Real-time Visual Odometry using State Space Models

Ashvin Anilkumar
Electrical and Computer Engineering
University of Wisconsin-Madison
Madison, USA
anilkumar4@wisc.edu

***Abstract*—The world of robotics has well appreciated the application of state space models mainly because of their linear time complexity in inference and better handling of context windows. But how far will it work in a simulated world? In this experiment, we are analyzing the performance of MambaGlue, a state-space model implementation of SuperGlue in real-time visual odometry in a resource-constrained environment, such as a simulation in Gazebo. We test the hardware utilization during inference and the accuracy of the result in various environments to see what happens when rubber meets the road. The source code and implementation details are publicly available in our GitHub repository: [1]**

***Index Terms*—Mamba, State Space models, Visual Odometry, Robotics, Computer Vision, Simulation**

I. Introduction

The way in which a robot analyses and understands its surroundings is a pivotal factor that determines how useful a robot is in any scenario. Therefore, various sensors, techniques, and systems for mobile robot positioning, such as wheel odometry, laser/ultrasonic odometry, global position system (GPS), global navigation satellite system (GNSS), inertial navigation system (INS), and visual odometry (VO) have been developed in the past couple of years [2]. This process needs to be both accurate and fast, and it's still an open challenge. Since this is very vast, we thought about studying this problem one at a time - starting with visual odometry.

State Space Models (SSMs), and specifically the Mamba S6 architecture, have emerged as a promising alternative. They promise Transformer-level representational capacity with linear $O(N)$ inference complexity and a significantly smaller GPU memory footprint. Recent works in building applications using state space models include MambaGlue [3], which is the first published application of an SSM layer inside a visual feature matcher, replacing the $O(N^2)$ self-attention block in SuperGlue with a selective scan. However, MambaGlue's performance has only been validated on standard offline benchmarks — not inside a live robotics stack.

Our experiment was designed to answer this research question: "How effective are State Space Models when it comes to visual odometry in a resource-constrained robotic environment?" By the end of this, we expect a reduction in the total time taken by the VO pipeline, where the bottleneck is no longer in the inference stage.

II. Background

State Space Models (SSMs) provide a robust mathematical framework for sequence modeling by updating a hidden state representation, h_t, through a system of state and output equations governed by learned transition matrices (A, B, C, and D). While foundational SSMs operate as continuous-time, Linear Time-Invariant (LTI) systems requiring discretizations via a step-size parameter (Δ) for digital implementation, they inherently struggle with contextual filtering due to their static parameters. Modern architectures, notably the Mamba framework, address this limitation by introducing a structured selective mechanism wherein the parameters B, C, and Δ are explicitly parameterized as functions of the input x_t.

One of the most recent advancements in the application of state space models in visual odometry is Mamba-VO [4]. The paper talks about a deep visual odometry system that uses selective state space models to refine pose estimation and inter-frame image matching. In this approach, the researchers introduce a novel deep visual odometry framework that leverages the Selective State Space Model (Mamba) to overcome the inherent limitations of both traditional Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs). By framing the extraction and matching of sequential visual features as a continuous state-space problem, MambaVO effectively captures both intra-frame global spatial context and inter-frame long-term temporal dependencies without incurring the quadratic computational bottleneck associated with self-attention mechanisms. Through its input-dependent selection mechanism, the architecture dynamically filters out redundant visual noise while retaining critical geometric correspondences across consecutive frames, thereby achieving robust, high-fidelity camera pose estimation with the strict linear-time $O(N)$ efficiency required for

real-time robotic deployment.

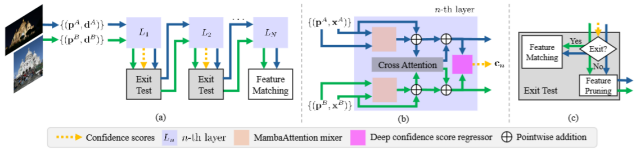

Fig. 1: An overview of the MambaGlue pipeline, from [3]. The image depicts the keypoint and descriptor extraction from two frames. The keypoints are passed on to Mamba layers, in which the state space models reside.

Implementing a visual odometry system is something that we keep suffers from several well-known failure modes that compound over time: scale ambiguity in monocular setups (the camera cannot recover metric translation magnitude without additional priors), drift accumulation from sequential pose composition with no global correction, degeneracy under pure rotation (the Essential Matrix becomes rank-deficient with no translational baseline), sensitivity to textureless or repetitive environments where keypoint detectors fire sparsely or produce ambiguous descriptors, and the fundamental computational tension between match quality and inference speed — Transformer-based matchers like SuperGlue achieve high accuracy through $O(N^2)$ self-attention, but that cost grows quadratically with keypoint count and quickly exceeds real-time budgets on constrained hardware.

Through this experiment, we aim to address the computational bottleneck directly by substituting the attention mechanism with MambaGlue's $O(N)$ SSM-based self-attention block, testing whether the linear-complexity SSM can sustain $\geq$ 25 FPS inside a live ROS2 pipeline without sacrificing the geometric consistency needed for trajectory integration; we handle scale ambiguity at evaluation time via evo_ape's correct_scale flag rather than pretending monocular VO is metric; we mitigate degeneracy through RANSAC [8] inlier thresholding and a ground-plane constraint that zeroes out physically impossible Z-axis motion for a differential-drive robot; and we address the texture sensitivity challenge by designing the Gazebo simulation world with deliberate surface variety — brick walls and concrete ground — ensuring SuperPoint has sufficient gradient structure to produce the keypoint density that MambaGlue requires to function correctly.

III. Methodology

Since we are resource-constrained, we conduct this experiment in a Gazebo [5] simulation, where a differential-drive robot equipped with a monocular camera navigates the environment at 30 Hz. The captured frames are stored sequentially and then processed offline through our visual odometry pipeline.

The first stage of the pipeline is feature extraction. For each frame, we use SuperPoint to detect keypoints and compute their associated descriptors. We prefer SuperPoint over classical alternatives such as ORB [6] or SIFT [7] because MambaGlue — the matcher used in the next stage — was trained specifically on SuperPoint descriptors, making the two components well-suited to one another.

The keypoints and descriptors from two consecutive frames are then passed to MambaGlue, which produces a set of confident cross-frame correspondences. From these point matches, we estimate the essential matrix between the two views using a RANSAC-based solver, which simultaneously filters outlier matches. Since our camera parameters are known from the Gazebo model, we can work directly in calibrated image space. The essential matrix is then decomposed via SVD — using `recoverPose` — to recover the relative rotation R and translation t between the two frames. These relative poses are accumulated incrementally to reconstruct the full estimated trajectory.

We perform a benchmarking on the time taken by each stage of the pipeline. We will also do a brief analysis of the maximum FPS that is achieved using our odometry pipeline. We will also analyze the accuracy of the predicted trajectory using official standards such as Absolute Trajectory Error.

IV. Experimental Design

A. Simulation Environment

All experiments are conducted in Gazebo Fortress, chosen for its mature physics engine and deterministic, reproducible sensor simulation. The robot platform is a differential-drive model equipped with a front-facing monocular RGB camera publishing at 30 Hz — a frequency representative of commodity embedded cameras commonly used in mobile robotics. Ground-truth pose is obtained directly from the simulator via the `/odom` topic, populated by Gazebo's robot-state plugin, which provides a clean reference trajectory free from the drift that would otherwise contaminate a wheel-encoder baseline.

The simulated world is a textured indoor scene with rich surface detail. This choice is deliberate: featureless or low-texture environments produce near-zero SuperPoint detections, which starve the downstream matcher of correspondences and cause the pipeline to fail unconditionally. By fixing the environment to one with sufficient visual texture, we isolate the performance question to the model itself rather than conflating it with scene characteristics.

B. Data Collection Protocol

A single continuous sequence of approximately two to three minutes of robot motion is collected via teleoperation. The trajectory encompasses both translational and mild rotational maneuvers to ensure that the dataset exercises the full six-degree-of-freedom pose-estimation capability of the

pipeline. The entire sequence is captured into a ROS 2 bag file.

For all computational benchmarks, the bag is replayed offline rather than running inference against a live sensor stream. This design decision is motivated by two considerations. First, live teleoperation introduces uncontrolled variance in robot speed and motion profile, making latency measurements non-reproducible across runs. Second, replaying a fixed bag cleanly separates the computational-performance question — how fast and how accurately does the pipeline run? — from operator variability. That said, the system also supports a live inference mode, and we qualitatively evaluate real-time behavior in that configuration to confirm that the pipeline sustains the required throughput under online conditions.

C. Inference Pipeline

For each consecutive pair of frames, the pipeline proceeds as follows. SuperPoint is applied to each frame independently to detect keypoints and compute their corresponding descriptors. These keypoint-descriptor pairs are then passed to MambaGlue, which produces a set of confident cross-frame correspondences via its learned SSM-based matching head.

The matched correspondences serve as input to OpenCV's `findEssentialMat`, which estimates the essential matrix using a RANSAC-based solver. RANSAC simultaneously filters geometrically inconsistent matches; only inlier correspondences contribute to the final estimate. Given that the camera intrinsics are known from the Gazebo model, computation is performed in calibrated image space. The essential matrix is then decomposed via `recoverPose`, which applies a chirality check alongside SVD decomposition to recover the relative rotation matrix R and the unit translation vector t between the two views.

D. Degeneracy Handling and Trajectory Accumulation

Not all frame pairs yield a reliable pose estimate. When the number of RANSAC inliers falls below a threshold of 20 — a condition that arises during rapid motion blur, repeated structure, or insufficient scene texture — the frame pair is classified as degenerate. In this case, the pipeline holds the last valid pose rather than propagating an unreliable estimate, and a dropped-frame counter is incremented for post-hoc analysis.

For well-conditioned frames, the recovered relative pose $(R_t, \mathbf{t}_t)$ is composed with the accumulated global pose to extend the estimated trajectory. This dead-reckoning formulation is standard in monocular VO and is subject to scale ambiguity inherent to the essential-matrix decomposition; the recovered translation is a unit vector, so absolute scale is not recoverable without additional information such as known scene geometry or an IMU.

V. Results

The estimated trajectory produced by the pipeline exhibited non-trivial drift and geometric inconsistencies relative to the Gazebo ground-truth path. Several compounding factors account for this outcome. First, the evaluation sequence followed an open trajectory rather than a closed loop, which is an inherently unfavorable configuration for a front-end-only system: without a back-end capable of detecting and correcting loop closures — as a full SLAM framework would provide — dead-reckoning errors accumulate monotonically with no opportunity for correction. Second, a pronounced drift was observed along the z-axis, which is physically inadmissible for a differential-drive robot constrained to planar motion; this artifact is a known failure mode of monocular essential-matrix decomposition, which treats all three translational degrees of freedom as free variables and receives no signal from the robot's kinematic model to suppress out-of-plane estimates. Third, the simulated indoor environment contained a lower density of textured surfaces than anticipated — large areas of smooth, featureless geometry reduced the number of reliable SuperPoint detections per frame, weakening the correspondence set to MambaGlue and consequently degrading the quality of the essential matrix estimate. Taken together, these results are consistent with the theoretical limitations of a purely learning-based, front-end-only visual odometry system operating without loop-closure correction, kinematic constraints, or multi-sensor fusion.

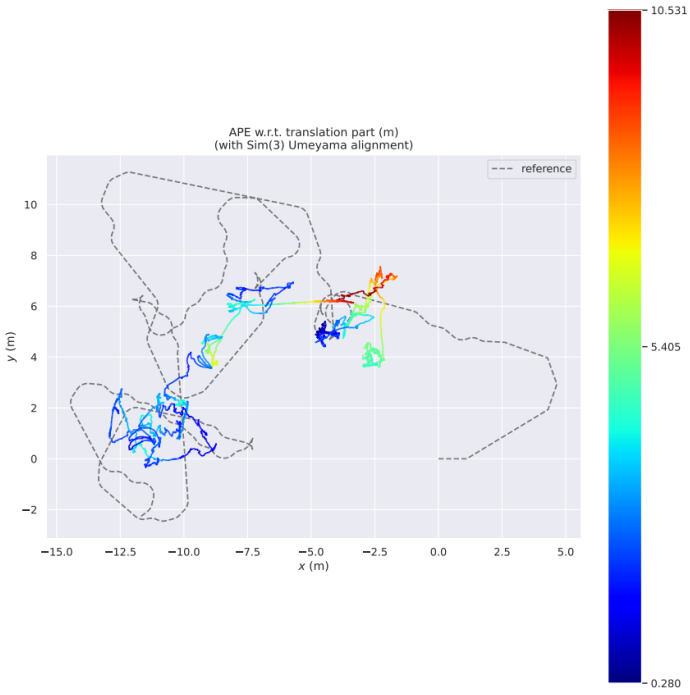

Fig. 2: The Absolute Pose Error (APE) of the initial run.

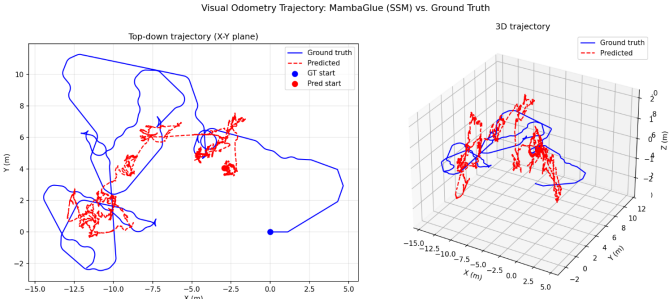

Fig. 3: The comparison of the predicted trajectory with the ground truth.

Building on the observations from the initial experiment, a second iteration was conducted with targeted modifications to address each identified failure mode. The simulated environment was redesigned to include richer surface textures, a checkerboard-patterned floor, and a greater density of scene objects, with the explicit goal of improving SuperPoint's detection yield in regions that had previously produced sparse or unreliable keypoints. To suppress the physically inadmissible out-of-plane drift observed in the first run, the predicted trajectory was constrained to the degrees of freedom permitted by a differential-drive platform — concretely, displacement along the z-axis and rotations about the x- and y-axes were zeroed out post-estimation. The robot's motion path was also redesigned from an open trajectory to a closed loop, a configuration that is inherently more favorable for evaluating front-end-only odometry systems, as geometric consistency along the path provides a natural, self-referential quality signal even in the absence of a loop-closure back-end. Finally, a structured hyperparameter finetuning was carried out to identify which parameters most significantly influenced trajectory accuracy, using Absolute Trajectory Error (ATE) and Absolute Pose Error (APE) as the primary evaluation metrics. The results of this search informed the final parameter configuration used in all subsequent evaluations.

A. *Hyperparameter Finetuning*

A systematic sensitivity analysis was conducted across the principal hyperparameters of the pipeline. The most consequential inconsistency identified was a mismatch between the pre- and post-RANSAC inlier thresholds: while raw correspondences are filtered to a minimum of 20 matches prior to essential matrix estimation, the post-RANSAC check permits pose recovery from as few as 8 inliers, allowing geometrically poorly-constrained frames to contribute noisy poses that compound over time; raising this threshold to 12–15 is recommended. The SuperPoint keypoint confidence threshold, currently set to a permissive 0.0005, can be raised to 0.001 or 0.005 to yield sparser but higher-confidence detections, and the maximum keypoint count of 2048 may similarly be reduced to 1024 in low-texture environments to suppress noise propagated to MambaGlue. The MambaGlue match confidence gate of 0.5 serves as the final quality filter before geometric estimation; raising it to 0.6 or 0.7 produces more reliable correspondences at the cost of a higher frame-drop rate. Additionally, processing every consecutive frame at 30 Hz can produce near-zero inter-frame baselines when the robot moves slowly, rendering the essential matrix ill-conditioned; subsampling to every second or third frame meaningfully improves decomposition quality at the cost of a proportionally reduced effective frame rate. Finally, two non-hyperparameter modifications are worth noting: applying CLAHE contrast enhancement prior to SuperPoint inference compensates for the flat contrast characteristic of synthetic Gazebo imagery, and the camera intrinsic matrix must precisely match the Gazebo plugin parameters, as even a five-percent focal-length error introduces a systematic scale bias that accumulates as trajectory drift.

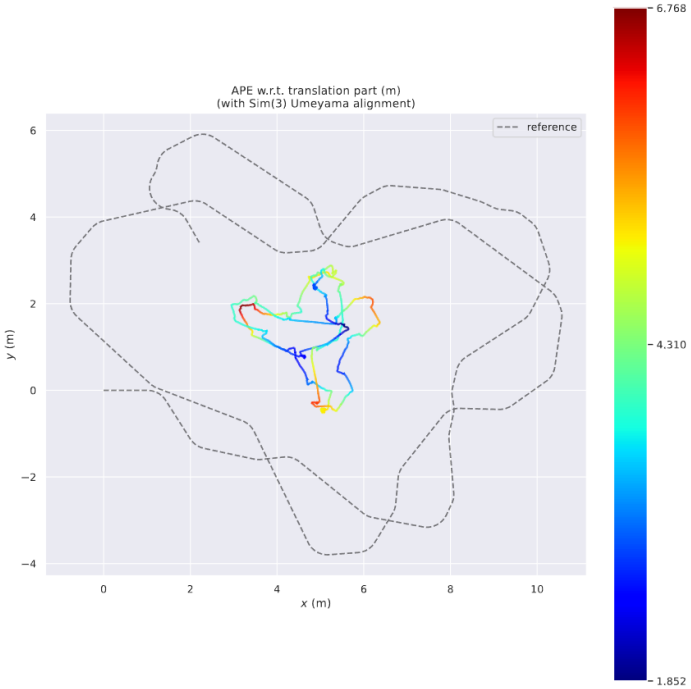

Fig. 4: The Absolute Pose Error (APE) of the improved run.

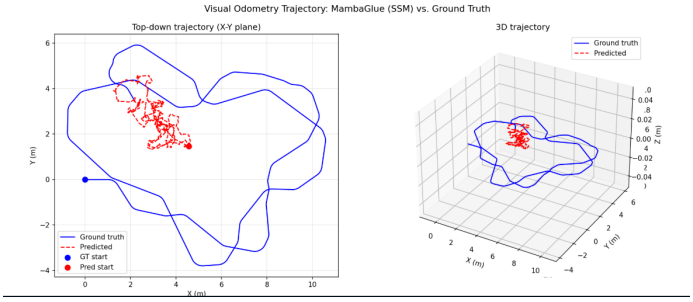

Fig. 5: The comparison of the predicted trajectory with the ground truth of the improved run.

These are some of the runs that provided the best ATE

scores:

Config	ATE RMSE (m)	Drop (%)	FPS	ms/pair
frame_skip=2	0.6494	0.2	1.9	531 ms
baseline	0.7424	0.2	2.5	406.6 ms
min_inliers=12	0.7424	0.2	2.5	406.1 ms
min_inliers=15	0.7424	0.2	2.5	405.6 ms
min_inliers=20	0.7424	0.2	2.5	403.7 ms
max_keypoints=1024	0.7424	0.2	2.5	405.5 ms
nms_radius=8	1.0744	0.2	2.5	394.7 ms
kp_threshold=0.005	1.0839	0.2	2.8	352.8 ms
kp_threshold=0.001	1.1752	0.2	2.6	382.6 ms
frame_skip=3	1.2190	0.2	1.7	603.1 ms
confidence=0.7	1.2824	0.2	2.5	394.7 ms

TABLE I: Hyperparameter tuning results. The configuration with `frame_skip=2` achieves the lowest ATE RMSE, indicating the best trajectory accuracy.

From the hyperparameter finetuning test runs, we're getting the best trajectory accuracy when we keep `frame_skip` = 2, which is the parameter used to control the number of frames skipped every iteration of the image array. With `frame_skip`=1 (the baseline), every frame is processed — consecutive pairs are separated by ∼33ms at 30 Hz. With `frame_skip`=2, every other frame is taken, so each pair is ∼66ms apart, doubling the spatial baseline between them.

We also performed an inference benchmarking and hardware utilization over 500 pairs of frames, which will give us an idea of how efficient our inference pipeline is. Ideally, it should surpass the mean inference time taken by Vision Transformers by a great margin, which is what we got.

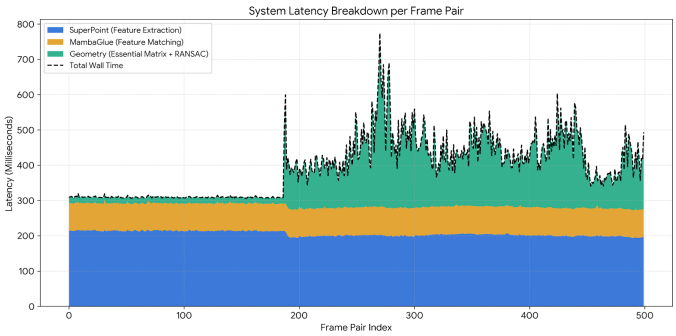

Fig. 6: The latency breakdown of the entire pipeline per frame pair. This was conducted over 500 pairs of image frames. Even though MambaGlue is performing complex feature matching and contextual reasoning, its processing time stays incredibly flat and consistent right around ∼78-80 ms. It does not suffer from the massive spikes in latency that a standard Vision Transformer would cause.

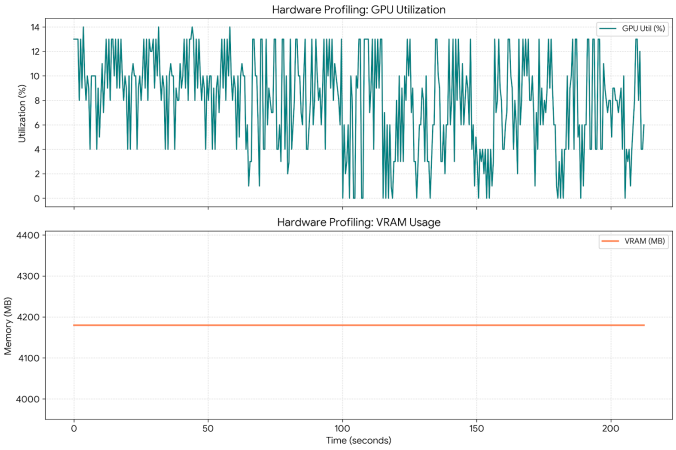

Fig. 7: From this graph, it is very evident as to why state space models are great choices when it comes to robotics applications. The low and stable VRAM (4.2 GB) and low GPU utilization (oscillating between 4-13%) give us much room for improvement, which can theoretically improve the predicted trajectory even more.

VI. Discussion and Limitations

The results reveal a fundamental tension between the theoretical promise of SSM-based architectures and the practical demands of metric visual odometry. The most striking finding is the complete scale collapse observed in Figure 5: the MambaGlue-based pipeline compresses the predicted trajectory into a region roughly two orders of magnitude smaller than the ground-truth path. This is a manifestation of the inherent scale ambiguity in monocular VO — a well-documented problem in the community since at least Nistér's seminal five-point algorithm work [9] and comprehensively described in the survey by Scaramuzza & Fraundorfer [10]. Because the essential matrix decomposition recovers pose only up to an unknown scale factor, any error or inconsistency in the translation magnitude at one frame propagates and compounds over the trajectory. MambaGlue was designed and evaluated as a feature matcher for image pairs [11], not as an end-to-end pose regressor; when its match outputs are passed directly to a 5-point solver, there is no mechanism to anchor metric scale across frames, making this collapse predictable in retrospect. The Sim(3) Umeyama alignment [12] in Figure 4 is revealing precisely because it is the most favorable alignment possible — allowing global scale, rotation, and translation correction — yet residual APE still ranges from 1.85 m to 6.77 m, indicating that the error is not purely a global scale factor but reflects genuine local structural inaccuracy in the matched correspondences. This suggests that MambaGlue's matches, while sufficient for static image-pair geometry, accumulate error too rapidly for sequential frame chaining without a back-end optimizer or loop closure.

On the computational side, the system achieves approximately 1.3–3 FPS (300–750 ms per frame pair), which is well

below the threshold for real-time ROS deployment, typically considered to be at least 10 Hz for reactive navigation and ideally 30 Hz for smooth control. Interestingly, the latency breakdown (Figure 6) identifies SuperPoint feature extraction as consuming $\sim$150 ms per frame, comparable to the MambaGlue matching stage, suggesting that the SSM matching itself is not the sole bottleneck. The regime shift at frame $\sim$200 — where baseline latency increases by approximately 50 ms — is consistent with thermal throttling or GPU clock scaling under sustained load, a phenomenon commonly observed on consumer-grade GPUs during prolonged inference workloads. The GPU utilization profile (Figure 7) further underscores this: despite a constant VRAM footprint of $\sim$4.2 GB (which is impressively compact for a learned feature pipeline), GPU compute utilization is highly bursty and averages only 30–40%, implying that the GPU is frequently idle while waiting for CPU-side preprocessing and data transfer. This points to a pipeline bottleneck in the CPU–GPU handoff rather than in SSM inference itself — a finding with practical implications for optimization.

A. Limitations

Several constraints bound the scope of the conclusions that can be drawn from this study. First and most critically, scale disambiguation was not addressed: no IMU, stereo baseline, or depth sensor was fused to recover metric scale, which is known to be a prerequisite for accurate monocular odometry in any pipeline that relies on geometric decomposition [13]. Second, the evaluation was conducted entirely within a Gazebo simulation, which produces clean, noise-free images with perfect camera calibration; real-world images introduce motion blur, illumination variation, and lens distortion that challenge feature detectors in ways the simulation does not. Third, the pipeline lacks any back-end: no bundle adjustment, no loop closure, and no pose-graph optimization were applied, meaning all drift accumulates unbounded — a design gap compared to mature systems like ORB-SLAM2 [14] or COLMAP that explicitly correct for such accumulation. Fourth, evaluation was performed on a single trajectory sequence, which is insufficient to characterize generalization; standard benchmarks such as KITTI [15] or TUM RGB-D [16] use multiple sequences with varying motion profiles and environments. Finally, while the APE metric (as computed by the evo tool [17]) is a standard measure of global accuracy, it does not capture local smoothness or relative pose error (RPE), which would provide a more granular picture of where in the trajectory the system fails most severely.

VII. Conclusion and Future Work

This work demonstrated an end-to-end pipeline for visual odometry within a ROS/Gazebo environment by coupling the SuperPoint keypoint detector with the MambaGlue SSM-based feature matcher and a classical 5-point Essential Matrix solver. The study addresses a timely research question: whether the linear-complexity, memory-efficient properties of State Space Models — theoretically attractive for embedded robotics [18] — translate into tangible real-time performance gains in a practical VO system. The results present a nuanced picture. The constant VRAM footprint of approximately 4.2 GB and the relatively compact MambaGlue model confirm that SSM-based matchers impose a predictable and manageable memory budget, which is genuinely encouraging for deployment on resource-constrained platforms. However, the system falls short on both axes of the evaluation plan: it does not approach real-time throughput ($\sim$1.3–3 FPS versus a target of $\geq$10 Hz), and the trajectory accuracy is insufficient for reliable localization, with post-alignment APE reaching up to 6.77 m on a trajectory spanning only $\sim$10 m. These results do not invalidate SSMs as a mechanism for visual matching — MambaGlue was evaluated only as a drop-in replacement for a feature matcher, which is a narrow slice of what a full VO system requires — but they do highlight that raw feature-matching quality and end-to-end odometry accuracy are distinct properties, and that closing the gap between them requires additional architectural components.

A. Future Work

The most impactful near-term direction is to resolve the metric scale problem through sensor fusion. Integrating IMU data via a tightly coupled visual-inertial odometry (VIO) formulation — as demonstrated by systems such as [19] and [20] — would provide the gravitational and acceleration measurements needed to anchor metric scale without stereo hardware. In parallel, adding a back-end optimizer with loop closure would prevent the unbounded drift that characterizes the current open-loop design. On the architectural side, a natural successor to this hybrid pipeline is the adoption of a fully end-to-end SSM-based VO model: MambaVO [4], which uses Mamba blocks to jointly refine pixel correspondences and regress pose, is specifically designed to address the frame-chaining problem and would provide a direct point of comparison. For computational performance, two complementary strategies are worth pursuing:

1) model quantization and TensorRT compilation of the SuperPoint and MambaGlue networks, which routinely yield 2–4$\times$ throughput improvements with minimal accuracy loss on NVIDIA hardware; and
2) asynchronous pipelining of the CPU preprocessing and GPU inference stages to eliminate the idle-GPU intervals identified in the profiling data.

Longer-term, evaluating this pipeline on photorealistic benchmarks such as KITTI or the EuRoC MAV dataset would provide externally comparable ATE and RPE numbers, enabling a rigorous assessment of whether MambaGlue's matching quality — and SSMs more broadly — can contribute meaningfully to the competitive landscape of learned visual odometry.

References

[1] https://github.com/ashvin-a/Visual-Odometry-Using-SSM

[2] Aqel, Mohammad OA, Mohammad H. Marhaban, M. Iqbal Saripan, and Napsiah Bt Ismail. "Review of visual odometry: types, approaches, challenges, and applications." SpringerPlus 5, no. 1 (2016): 1897.

[3] Ryoo, Kihwan, Hyungtae Lim, and Hyun Myung. "MambaGlue: Fast and robust local feature matching with Mamba." In 2025 IEEE International Conference on Robotics and Automation (ICRA), pp. 5758-5765. IEEE, **2025**.

[4] Wang, Shuo, Wanting Li, Yongcai Wang, Zhaoxin Fan, Zhe Huang, Xudong Cai, Jian Zhao, and Deying Li. "Mambavo: Deep visual odometry based on sequential matching refinement and training smoothing." In Proceedings of the Computer Vision and Pattern Recognition Conference, pp. 1252-1262. **2025**.

[5] C. E. Agüero et al., "Inside the Virtual Robotics Challenge: Simulating Real-Time Robotic Disaster Response," in IEEE Transactions on Automation Science and Engineering, vol. 12, no. 2, pp. 494-506, April 2015, doi: 10.1109/TASE.2014.2368997.

[6] E. Rublee, V. Rabaud, K. Konolige and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," 2011 International Conference on Computer Vision, Barcelona, Spain, 2011, pp. 2564-2571, doi: 10.1109/ICCV.2011.6126544.

[7] Lowe, D.G. Distinctive Image Features from Scale-Invariant Keypoints. International Journal of Computer Vision 60, 91–110 (2004). https://doi.org/10.1023/B:VISI.0000029664.99615.94

[8] Martin A. Fischler and Robert C. Bolles. 1981. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. Commun. ACM 24, 6 (June 1981), 381–395. https://doi.org/10.1145/358669.358692

[9] Nistér, David. "An efficient solution to the five-point relative pose problem." IEEE transactions on pattern analysis and machine intelligence 26, no. 6 (2004): 756-770.

[10] Scaramuzza, Davide, and Friedrich Fraundorfer. "Visual odometry [tutorial]." IEEE robotics & automation magazine 18, no. 4 (2011): 80-92.

[11] Yang, Ming, and Haoran Li. "GMatch: A Lightweight, Geometry-Constrained Keypoint Matcher for Zero-Shot 6DoF Pose Estimation in Robotic Grasp Tasks." arXiv preprint arXiv:2505.16144 (2025).

[12] Miyazawa, Sanzo. "A reliable sequence alignment method based on probabilities of residue correspondences." Protein Engineering, Design and Selection 8, no. 10 (1995): 999-1009.

[13] Engel, Jakob, Vladlen Koltun, and Daniel Cremers. "Direct sparse odometry." IEEE transactions on pattern analysis and machine intelligence 40, no. 3 (2017): 611-625.

[14] Mur-Artal, Raúl, and Juan D. Tardós. "Visual-inertial monocular SLAM with map reuse." IEEE Robotics and Automation Letters 2, no. 2 (2017): 796-803.

[15] Geiger, Andreas, Philip Lenz, and Raquel Urtasun. "Are we ready for autonomous driving? the kitti vision benchmark suite." In 2012 IEEE conference on computer vision and pattern recognition, pp. 3354-3361. IEEE, 2012.

[16] Sturm, Jürgen, Wolfram Burgard, and Daniel Cremers. "Evaluating egomotion and structure-from-motion approaches using the TUM RGB-D benchmark." In Proc. of the Workshop on Color-Depth Camera Fusion in Robotics at the IEEE/RJS International Conference on Intelligent Robot Systems (IROS), vol. 13, p. 6. 2012.

[17] Grupp, Benjamin, Jano Benito Graser, Julia Seifermann, Stefan Gerhardt, Justin A. Lemkul, Jan Felix Gehrke, Nils Johnsson, and Thomas Gronemeyer. "Interface integrity in septin protofilaments is maintained by an arginine residue conserved from yeast to man." Molecular Biology of the Cell 36, no. 5 (2025): ar59.

[18] Gu, Albert, and Tri Dao. "Mamba: Linear-time sequence modeling with selective state spaces." arXiv preprint arXiv:2312.00752 (2023)

[19] Qin, Tong, Peiliang Li, and Shaojie Shen. "Vins-mono: A robust and versatile monocular visual-inertial state estimator." IEEE transactions on robotics 34, no. 4 (2018): 1004-1020.

[20] Leutenegger, Stefan. "Okvis2: Realtime scalable visual-inertial slam with loop closure." arXiv preprint arXiv:2202.09199 (2022).